

Software Development Life Cycle (SDLC) Specification & Reference Guide

Repository: EMAIL-AGENT-v1

Document Level: Authoritative Engineering & Operational Reference

Last Aligned: 2026-08-25

Primary Authorities: AGENTS.md , README.md , tests/README.md , pyproject.toml ,
.github/workflows/ci.yml , docs/architectures/TARGET-ARCHITECTURE.md ,
docs/evaluations/ , evaluations/HARNESS-GUIDE.md

Table of Contents

1. Executive Overview & System Architecture Context
2. Phase 1: Inception, Specifications & Architectural Governance
3. Phase 2: Local Development & Toolchain Standards
4. Phase 3: Verification & Test Harness Architecture
5. Phase 4: Offline Evaluation & Benchmarking Lifecycle
6. Phase 5: Quality Gates, Pre-PR Verification & CI/CD Pipeline
7. Phase 6: Deployment, Persistence & Runtime Modes
8. SDLC Lifecycle Summary Matrix & Comprehensive Workflow Diagram

1. Executive Overview & System Architecture Context

The EMAIL-AGENT-v1 codebase represents an enterprise-grade AI coworker system (*Cowork Agent*) designed around **two strictly decoupled product flows** operating over a unified control plane and persistence engine (AGENTS.md:L15-25 , README.md:L9-52 , docs/architectures/current-architectures/04-overall-architecture.md).

leaking email bodies into chat context ([AGENTS.md:L23](#) , [README.md:L50](#) , [TARGET-ARCHITECTURE.md:L41](#)).

1.2 Strict Layered Architecture & Dependency Direction

The codebase enforces a strict directional architecture where dependencies point strictly inward toward domain contracts ([AGENTS.md:L43-45](#)):

domain ← features ← (integrations | orchestration | persistence) ← app (FastAPI Root)

```
src/cowork_agent/
├── domain/                # Pure domain models (frozen dataclasses, no fram
├── features/              # Core business logic
│   ├── ai_chat/           # Chat controller, memory gateway, intent routing
│   ├── batch_evaluation/  # Scalable batch eval plugins & credential leasin
│   ├── email_action_plan/ # Email digest workflows, policies, ports, schema
│   └── user_documents/    # Ingestion state machine and extraction pipeline
├── integrations/          # External systems & hardware adapters
│   ├── gmail/             # Google OAuth & Gmail API adapters
│   ├── outlook/           # Microsoft Graph OAuth & Mail API adapters
│   ├── llm/               # Provider implementations (Gemini, Mistral, Open
│   ├── knowledge_ingestion/ # PdfInspector & DocxExtractor parsers
│   └── rag/               # Hybrid Turbovec, BM25, RRF fusion, embeddings &
├── orchestration/         # Background workers (dev server, mail-todo-worke
├── persistence/          # PostgreSQL & SQLite repositories, migrations (0
├── app.py                 # FastAPI composition root & entry point (mail-to
├── config.py              # Environment settings loaders & fallback resolut
└── prompting.py           # Shared block delimiters preventing prompt injec
```

2. Phase 1: Inception, Specifications & Architectural Governance

The inception and architectural evolution of any feature or modification in [EMAIL-AGENT-v1](#) follow an immutable governance protocol rooted in PRDs, ADRs, and capability specifications.



2.1 PRD Hierarchy (Product Requirements)

Located in `tasks/prds/`, PRDs define user intent, functional requirements (FR), and acceptance criteria across product versions:

- `PRD-v0-product_requirements.md` : Baseline email ingestion and action plan synthesis MVP.
- `PRD-v1-Core-Email-and-RAG.md` : Core single-turn Email RAG, intent routing (`NO_ACTION` , `DIRECT_PLAN` , `RETRIEVE_RAG`), and company knowledge grounding.
- `PRD-v2-Memory-Extension.md` : 4-tier typed memory extension, chat session persistence, and episodic memory governance.
- `PRD-v3-chat-with-user-documents.md` : Grounded user document upload (PDF/DOCX), page-level citations, and intent classifier routing.
- `PRD-v4-pluggable-hybrid-rag-providers.md` : Pluggable vector store architecture and Turbovec transition.

2.2 ADR Lifecycle & Strict Decision Governance

Every non-trivial architectural change is recorded in an Architecture Decision Record under `tasks/adr/` (Note: this repository does **not** use `docs/adr/` , as mandated by `docs/agents/domain.md:L9`).

Key active ADRs governing system invariants:

- **ADR-001**: Async pipeline and adapter pattern for decoupled mail ingestion.
- **ADR-003**: Deferral of raw attachment processing (presence recorded only; content ignored).
- **ADR-004**: Chat-native `TaskEpisode` lifecycle and strict decoupling from `@Email` tool in chat.
- **ADR-006**: Supabase-managed data isolation with Gmail cookie sessions (no Supabase keys in browser).
- **ADR-007**: Project-scoped classifier-gated user document plane (`tenant → user → project → documents`).
- **ADR-008**: Turbovec vector store + PostgreSQL FTS for project-scoped document retrieval.
- **ADR-009**: Retirement of legacy Qdrant backend in favor of Turbovec with graceful degrade-to-null memory.
- **ADR-010**: Local PostgreSQL control plane for low-latency dev loops without porting memory to SQLite.
- **ADR-011**: Reply-chain context aggregation for conversational email threads.

- **ADR-012:** OpenRouter as last-resort fallback for Google Gemini LLM key exhaustion.

IMPORTANT

ADR Conflict Rule ([docs/agents/domain.md:L33-38](#)): If an implementation proposal or output contradicts an existing ADR, it must be explicitly surfaced (e.g., "*Contradicts ADR-004 — but worth reopening because...*") rather than silently overriding repository architecture.

2.3 Capability Specifications & Domain Modeling

- **Capability Specs** ([tasks/specs/](#)): Bridge PRDs and implementation. Notable specs include [SPEC-chat-ragas-evaluation.md](#) , [SPEC-memory-evaluation.md](#) , and [SPEC-parallel-memory-evaluation.md](#) .
- **Domain Modeling & Ubiquitous Language** ([docs/agents/domain.md:L27-32](#)): Developers and agents must strictly adhere to the domain glossary. Inventing synonyms for established domain entities is treated as a drift defect.

3. Phase 2: Local Development & Toolchain Standards

The repository enforces strict environment isolation, static type safety, and code formatting standards across both Python backend and TypeScript frontend.



3.1 Python Toolchain & Interpreter Isolation (Astral `uv`)

- **Virtual Environment Management:** All Python dependencies must be installed via `uv` ([AGENTS.md:L4-5](#)):

```
uv sync --extra dev --extra postgres
```

- **Strict Execution Directive** ([AGENTS.md:L49-50](#)): Always prefix Python commands with `uv run` . Bare `python` or `python -m` picks up the Windows Anaconda interpreter / App Execution Alias and crashes with unrelated SSL certificate or encoding errors.
- **Static Analysis & Strict Typing:**
 - `Mypy` (`strict = true` , `python_version = "3.13"`) ([pyproject.toml:L96-104](#)):

```
uv run mypy src
```

- **Ruff Linter & Formatter** (`line-length = 100` , rules: `["E", "F", "I", "UP", "B"]`) ([pyproject.toml:L88-95](#)):

```
uv run ruff check .
```

3.2 Frontend Toolchain (React 19 + TypeScript + pnpm)

- **Package Manager:** `pnpm@9.0.0` with Node `>=20` ([frontend/package.json:L5-8](#)).
- **Core Stack:** React `19.2.7` , Vite `8.1.1` , Tailwind CSS `4.3.3` , `@base-ui/react` , `@testing-library/react` , Vitest `4.1.10` , TypeScript `~6.0.2` ([frontend/package.json:L18-55](#)).
- **Frontend Verification Scripts:**

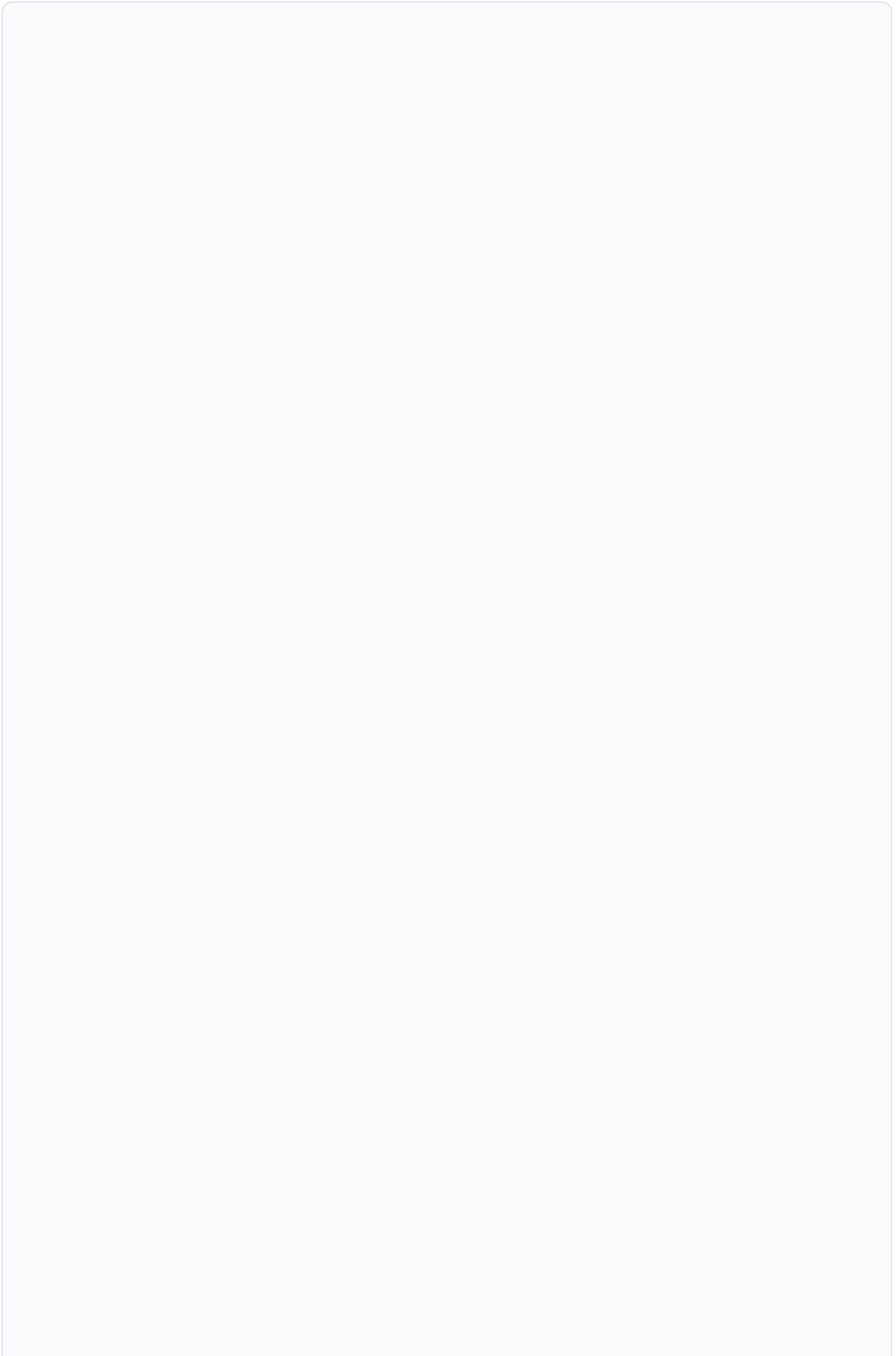
```
cd frontend
pnpm install
pnpm lint          # eslint .
pnpm check-types   # tsc -b --noEmit
pnpm test          # vitest run
pnpm build         # tsc -b && vite build
```

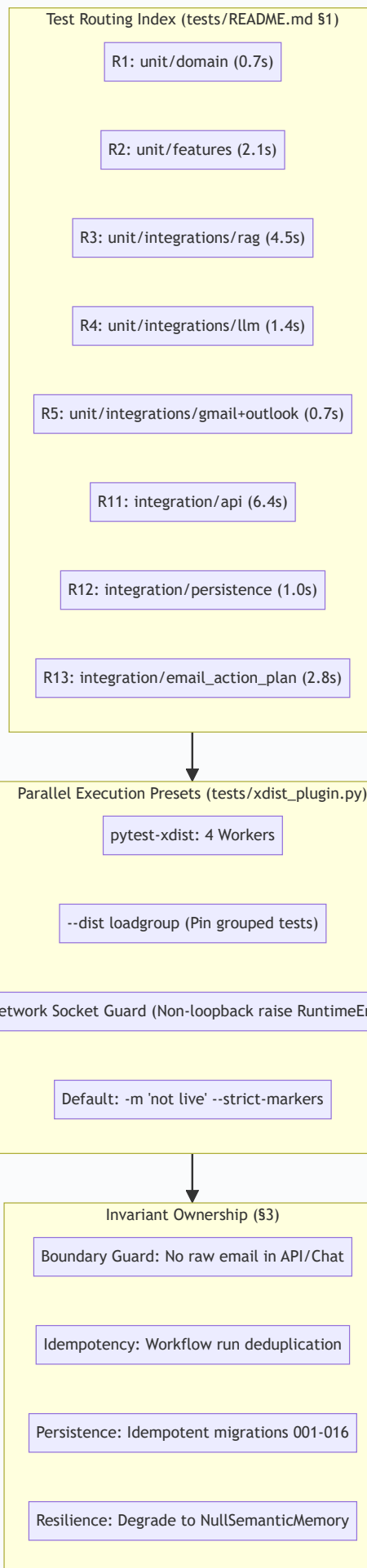
3.3 Workspace Privacy, Security & Isolation Rules

1. **Git Isolation via** `.git/info/exclude` : Personal scripts, scratch artifacts, and local config overrides must be kept out of git tracking via `.git/info/exclude` to prevent leaking unapproved files to teammates upon `git push` (`<RULE[user_global]>` , [agent-experience-registry.md:L98-103](#)).
2. **Zero-Secret Invariant:** Never commit `.env` files or secret keys (`TOKEN_ENCRYPTION_KEY` , `OAuth_STATE_SECRET` , `GEMINI_API_KEY` , `JINA_API_KEY`). Never expose server secrets in frontend `VITE_*` environment variables ([AGENTS.md:L65](#) , [TARGET-ARCHITECTURE.md:L228-229](#)).
3. **Transient Email Rule:** Raw email bodies and attachments are transient ephemeral memory only; they must never be stored in persistent SQL tables or vector embeddings ([AGENTS.md:L66-67](#)).

4. Phase 3: Verification & Test Harness Architecture

The testing philosophy of `EMAIL-AGENT-v1` is **Offline by Construction**, structured to run the entire suite of 1,600+ tests in under ~15–18 seconds on local hardware ([tests/README.md:L8](#)).





4.1 Granular Test Route Index (R1–R16)

Instead of executing the entire test suite on every minor change, developers and subagents pick the **narrowest covering route** from [tests/README.md:L14-58](#) :

Route	Target Path	Tests	Cost (Serial)	Architectural Scope Covered
R1	tests/unit/domain	179	0.7 s	Pure domain models, frozen contracts, enums, validation rules. Zero I/O.
R2	tests/unit/features	588	2.1 s	Chat controller, memory gateway, intent routing, action-plan mapping. Fakes only.
R3	tests/unit/integrations/rag	102	4.5 s	BM25, RRF fusion, reranker, query guard, key rotation, Turbovec memory.
R4	tests/unit/integrations/llm	77	1.4 s	Prompt assembly, parsing, key rotation, classifiers, OpenRouter last-resort.
R5	tests/unit/integrations/gmail + mailbox + outlook	54	0.7 s	Gmail/Microsoft OAuth, PKCE, token cipher, provider router, mailbox adapters.
R6	tests/unit/integrations	375	6.2 s	R3 + R4 + R5 plus bootstrap, Supabase adapters.
R7	tests/unit/persistence	37	1.8 s	Repository logic against deterministic in-memory fakes.
R8	tests/unit/orchestration	19	1.7 s	Background worker dispatchers, pollers, queue recovery.
R9	tests/unit/scripts	188	8.3 s	scripts/*.py evaluation CLIs (in-process main(argv)).
R10	tests/unit/fixtures	33	2.2 s	Golden fixture schemas, corpus frontmatter, and label validation.
R11	tests/integration/api	61	6.4 s	FastAPI routes via in-process ASGI transport (httpx.ASGITransport).
R12	tests/integration/persistence	9	1.0 s	Real PostgreSQL repositories (pg-control-plane xdist group; skips without DB).
R13	tests/integration/email_action_plan	38	2.8 s	Provider-neutral mailbox → classify → plan → persist on fakes.
R14	tests/integration	100	7.8 s	R11 + R12 + R13 plus corpus-backed end-to-end workflows.
R15	tests/unit	1538	14.1 s	Entire unit test surface.
R16	tests/unit --ignore=tests/unit/scripts	1350	9.1 s	Fast unit test tier (excluding evaluation CLI harnesses).
Full	uv run pytest -q	1638	~15 s (Parallel)	Full repository test suite.

4.2 Invariant Ownership Principle (§3 `tests/README.md`)

To prevent duplicate test maintenance and testing bloat, each critical system invariant is assigned a **single owner file** (`tests/README.md:L71-110`):

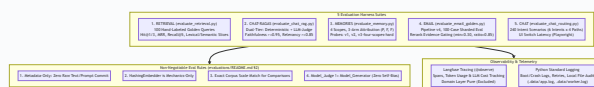
- **Raw Email Privacy Boundary:** Owned by `tests/integration/api/test_principal_boundary.py` (API level) and `tests/unit/domain/test_chat_contracts.py` (Chat memory level). Prohibited from being re-asserted in repository/gateway tests.
- **Run Idempotency:** Owned by `tests/integration/email_action_plan/test_workflow.py`.
- **Postgres Migrations Idempotency:** Owned by `tests/integration/persistence/test_postgres_repositories.py`.
- **Network Socket Guard:** Owned by `tests/unit/test_network_guard.py` (raises `RuntimeError` if any non-loopback outbound socket is opened during unit tests).
- **Graceful RAG Degradation:** Owned by `tests/unit/integrations/test_bootstrap.py`.

4.3 Parallel Execution & Hardware Optimization

- **Hardware Profile:** Optimized for multi-core architectures (e.g. Intel Core Ultra 7 255H with 16 cores: 6P + 8E + 2LP-E) (`<RULE[user_global]>`).
- **Worker Configuration:** Pytest defaults to 4 workers via custom `tests/xdist_plugin.py` using `--dist loadgroup` (or `--dist loadfile`) to prevent LP-E core stragglers and database schema contention (`tests/README.md:L6-10` , `pyproject.toml:L58-86`).
- **Single Consolidated Suite Audit:** As documented in `docs/evaluations/TEST-SUITE-AUDIT-REPORT.md:L81-118` , the suite abolished the separate `@pytest.mark.extended` tier. Splitting the suite deselected 382 production unit tests saving only 1.5s while risking silent regressions. The active standard is **one single comprehensive suite** running in ~15s.

5. Phase 4: Offline Evaluation & Benchmarking Lifecycle

The repository maintains an extensive evaluation framework across 5 distinct domains (`evaluations/README.md:L9-18` , `evaluations/HARNESS-GUIDE.md:L7-40`).



5.1 The 5 Evaluation Suites

1. RETRIEVAL Evaluation ([evaluations/RETRIEVAL/](#)):

- **Harness:** `scripts/evaluate_retrieval.py` against `tests/fixtures/rag/retrieval_golden.json` and `data/extracted/`.
- **Metrics:** `Hit@1`, `Hit@3`, `MRR`, `Recall@5` (document and section level), evaluated across 3 query slices (`lexical`, `semantic`, `mixed`), plus `abstention_rate`.
- **Execution:**

```
uv run python scripts/evaluate_retrieval.py --embedder gemini --retriever hy
```

2. CHAT-RAGAS Dual-Tier Evaluation ([docs/evaluations/RAGAS.md](#), [evaluations/CHAT-RAGAS/](#)):

- **Harness:** `scripts/evaluate_chat_rag.py --ragas`.
- **Tier 1 (Deterministic/Offline):** Page `Hit@1`, `Hit@5`, `MRR`, citation subset validation (`cited_pages ⊆ retrieved_pages`), and 3-stage latency breakdown (`retrieval`, `generation`, `evaluator`).
- **Tier 2 (LLM Judge):** Measures `faithfulness` (≥ 0.95) and `answer_relevancy` (≥ 0.85).
- **Strict Invariant:** `\text{model_judge} \neq \text{model_generator}` with `temperature = 0` to prevent self-preference bias ([docs/evaluations/RAGAS.md:L45-58](#)).

3. MEMORIES 4-Tier Memory Evaluation ([evaluations/MEMORIES/](#)):

- **Harness:** `scripts/evaluate_memory.py` using probe sets (`v1-four-scopes`, `v2-four-scopes-wide`, `v3-four-scopes-hard`).
- **3-Arm Attribution Protocol (P, F, F):** Full memory must Pass (P), Ablated memory must Fail (F), and Empty baseline must Fail (F) to prove earned memory attribution ([evaluations/MEMORIES/README.md:L28](#)).
- **Backend Support:** Full parity between SQLite (`POSTGRES_MODE=off`) and PostgreSQL (`POSTGRES_MODE=local` / `cloud`).

4. EMAIL Pipeline Evaluation ([evaluations/EMAIL/](#)):

- **Harness:** `scripts/evaluate_email_golden.py` (sharded across 50 cases/run) against `evaluations/EMAIL/golden_dataset.json`.
- **Metrics:** Actionability accuracy, retrieve-first compliance, Cohere evidence gate status (`EMAIL_RAG_MIN_RERANK_SCORE=0.30`, `EMAIL_RAG_RELATIVE_CUTOFF_RATIO=0.85`), and final route resolution.

5. CHAT Intent & Latency Evaluation ([evaluations/CHAT/](#)):

- **Harness:** `scripts/evaluate_chat_routing.py` (240-scenario dataset across 6 intents $\times$ 4 paths) and `e2e/chat-history-latency.spec.ts` for UI switch latency

benchmarking.

5.2 Four Non-Negotiable Evaluation Rules ([evaluations/README.md:L21-27](#))

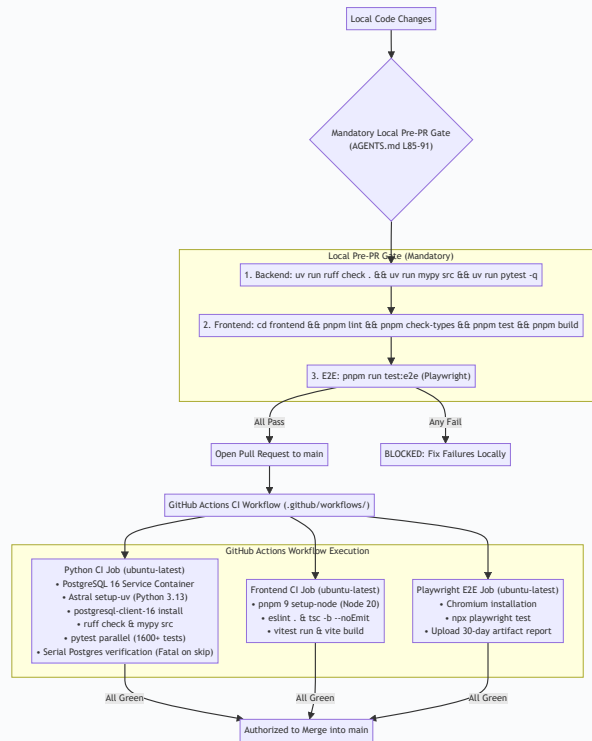
1. **Metadata-Only Commit:** Never commit raw email text, chat messages, prompts, or chunk text to git. Artifacts store only IDs, counts, verdicts, and timings.
2. **Mechanics-Only Hashing:** `--embedder hashing` / `--dry-run` is strictly for offline pipeline verification and carries zero semantic rank value.
3. **Corpus Scale Parity:** Evaluation reports are comparable only when executed against identical corpus chunk counts and probe set versions.
4. **Independent Judge:** Model judge must never equal model generator in LLM-as-a-judge evaluations.

5.3 Observability Architecture (Langfuse & Python Logging)

- **AI Telemetry (Langfuse Python SDK)** ([docs/observability/LANGFUSE.md](#)): Uses `@observe` across controllers, LLM providers, and RAG retrievers to track execution spans, tokens, and cost. Domain models are strictly excluded from importing Langfuse (`domain` remains pure python).
- **System Infrastructure Logging** ([docs/observability/PYTHON_LOGGING.md](#)): Uses Python `logging` with `StreamHandler` and `FileHandler` (`.data/app.log` , `.data/worker.log`) for server boot, unhandled exceptions, and transport retries.

6. Phase 5: Quality Gates, Pre-PR Verification & CI/CD Pipeline

The repository implements a strict, multi-tiered verification gate before any code can enter the `main` branch.



6.1 Mandatory 3-Step Local Pre-PR Gate

Before opening any Pull Request to `main`, developers and AI agents must run and pass the 3-step verification gate ([AGENTS.md:L85-91](#) , [tests/README.md:L124-138](#)):

```
# Step 1: Backend CI quality gate
uv run ruff check . && uv run mypy src && uv run pytest -q

# Step 2: Frontend CI quality gate
cd frontend && pnpm lint && pnpm check-types && pnpm test && pnpm build

# Step 3: End-to-End Playwright test suite
pnpm run test:e2e
```

6.2 GitHub Actions Workflows

- ci.yml (Python & Frontend Pipelines)** ([.github/workflows/ci.yml](#)):
 - Postgres Service Container:** Spins up `postgres:16-alpine` on port 5432 with healthchecks.
 - Client Parity:** Installs `postgresql-client-16` on Ubuntu runner to ensure `pg_dump` version matching for backup/restore tests ([.github/workflows/ci.yml:L51-59](#)).
 - Fast Parallel Test Suite:** Runs `uv run pytest -q --ignore=tests/integration/persistence`.

- **Fatal Skip Guard** ([.github/workflows/ci.yml:L71-79](#)): Runs the PostgreSQL persistence tier serially (`-n 0`) and fails the build with fatal error if any persistence test is skipped.
 - **Frontend Verification**: Runs frozen lockfile installation, `pnpm lint`, `pnpm check-types`, `pnpm test`, and `pnpm build` under Node 20.
2. **playwright.yml (E2E Integration)** ([.github/workflows/playwright.yml](#)):
- Installs Chromium browser dependencies and executes `npm playwright test`.
 - Uploads test execution reports with a 30-day artifact retention policy.

6.3 Agent Experience & Collaboration Patterns

Recorded in [docs/references/agent-experience-registry.md](#), the **MindSeeds Epistemic Compression Protocol** distills operational wisdom into load-bearing seeds (<12 words) that govern agent pair programming:

- *"Test narrow, prove broad"* ([agent-experience-registry.md:L55-61](#)): Use targeted unit tests first; expand only when contracts change.
- *"Fresh passing evidence needs no echo"* ([agent-experience-registry.md:L62-68](#)): Do not rerun passing test suites without intervening edits.
- *"Default pytest already fans out to 4 workers"* ([agent-experience-registry.md:L78-84](#)): Do not inject `-n 0` into standard test commands.
- *"An ignore rule is only real when git confirms it"* ([agent-experience-registry.md:L97-103](#)): Verify gitignore rules via `git check-ignore`.

7. Phase 6: Deployment, Persistence & Runtime Modes

The control plane supports three runtime persistence modes selected via `POSTGRES_MODE` in `.env` ([config.py:L41-80](#), [ADR-010](#), [current-architectures/03-control-plane-persistence-and-uis.md](#)).



7.1 Persistence Mode Decision Matrix

Dimension	POSTGRES_MODE=off	POSTGRES_MODE=local	POSTGRES_MODE=cloud
Target Engine	Local SQLite (<code>.data/*.db</code>) + In-process buffer	Local Docker <code>postgres:16-alpine</code>	Hosted Supabase PostgreSQL
Connection URL	<code>""</code> (Empty string in <code>config.py</code>)	<code>DATABASE_URL_LOCAL</code> (Default: <code>postgresql://cowork:cowork_dev_only@127.0.0.1:5432/cowork</code>)	<code>DATABASE_URL_CLOUD</code> (Direct/Session :5432)
Migrations	None (SQLite table creation on boot)	Migrations <code>001 – 016</code> applied on boot	Migrations <code>001 – 016</code> applied on boot
Outlook Support	Enabled (SQLite-linked connector)	Disabled (SQLite-only feature)	Disabled (SQLite-only feature)
Worker Support	In-process dispatching	<code>mail-todo-worker</code> enabled	<code>mail-todo-worker</code> enabled
Primary Use Case	Fast local dev loops, zero-dependency offline runs	Local MVP with low latency and full SQL parity	Deployed staging and production environments

7.2 Database Migrations (001–016)

PostgreSQL schema evolution is managed via SQL migrations in

`src/cowork_agent/persistence/migrations/`. Each migration ships both `.sql` (up) and `.down.sql` (rollback):

- `001_mail_todo.sql` : Mailbox connections, digest runs, task storage, outbox events.
- `002_chat_profiles.sql` – `004_task_episodes.sql` : Typed memory (declarative profiles, summaries, chat-native task episodes).
- `005_identity_workspace_sessions.sql` & `006_durable_chat_sessions.sql` : Tenant/workspace sessions and chat session registry.
- `007_projects_documents.sql` – `012_project_document_chunks.sql` : Project-scoped document storage, chunks, and FTS indexes.
- `014_chat_turn_lifecycle.sql` – `016_chat_turn_activity.sql` : Chat turn lifecycle, supersedes episode linking, and reasoning activity.

Migrations are applied automatically on FastAPI startup via `apply_migrations(pool)` guarded by PostgreSQL advisory locks (`config.py:L31-80` , `.github/workflows/ci.yml:L34-36`).

7.3 System Entry Points & CLI Commands

Defined in `pyproject.toml:L49-54` and `README.md:L158-171` :

- `mail-todo-api` : Starts the core FastAPI server (`uv run mail-todo-api` on port `8000`).
 - `mail-todo-worker` : Runs the background document ingestion queue worker.
 - `mail-todo-dev` : Concurrently boots both FastAPI API and background worker for development.
 - `mail-todo-ingest-knowledge` : Ingestion CLI converting raw company documents into committed Markdown in `data/extracted/` .
 - Frontend Dev Server: `cd frontend && pnpm dev` (Vite dev server on port `5173`).
-

8. SDLC Lifecycle Summary Matrix & Comprehensive Workflow Diagram

8.1 Summary Lifecycle Matrix

SDLC Phase	Core Artifacts & Tools	Primary Authority Files	Validation & Quality Gates
Phase 1: Inception & Governance	PRDs, Capability Specs, ADRs	tasks/prds/ tasks/specs/ tasks/adr/ docs/agents/domain.md	• Domain glossary alignment • ADR conflict check • Target architecture alignment
Phase 2: Local Dev & Tooling	uv , Python 3.13, pnpm 9 , React 19, Vite, Tailwind 4	AGENTS.md pyproject.toml frontend/package.json	• Strict typing (mypy src) • Linting (ruff check . , eslint .) • Type checking (tsc -b --noEmit)
Phase 3: Verification & Harness	Pytest, tests/xdist_plugin.py , Routes R1–R16	tests/README.md tests/conftest.py docs/evaluations/	• Route Index execution • Single Invariant Owner rule (§3) • Network socket isolation guard
Phase 4: Offline Evaluation	5 Evaluation Harnesses, Langfuse, Python Logging	evaluations/HARNESS-GUIDE.md evaluations/README.md docs/observability/	• Metadata-only commit rule • RAGAS judge $\neq$ generator • 3-arm memory attribution (P, F, F)
Phase 5: Quality Gates & CI/CD	Pre-PR script, GitHub Actions, Playwright E2E	AGENTS.md .github/workflows/ci.yml playwright.yml	• Mandatory 3-step local Pre-PR gate • CI Python (Postgres container + check) • CI Frontend & Playwright E2E
Phase 6: Persistence & Deployment	Docker Compose, Postgres migrations (001–016), FastAPI	config.py docker-compose.yml src/cowork_agent/persistence/	• Advisory lock migrations on boot • Fallback persistence verification • Service healthchecks

8.2 End-to-End SDLC Lifecycle Flow

